ВВЕДЕНИЕ В ENTITYFRAMEWORK CORE

Введение в EntityFramework Core

Тема лекции

Введение в EntityFramework Core

Цели и задачи

Изучить основы вреймворка EntityFramework Core

Научиться использовать контекст базы данных для формирования запросов.

Общая информация

ВВЕДЕНИЕ В ENTITYFRAMEWORK CORE

Введение в EntityFramework Core

Entity Framework (EF) Core - это облегченная, расширяемая кроссплатформенная версия популярной технологии доступа к данным Entity Framework с открытым исходным кодом.

Введение в EntityFramework Core

EF Core представляет собой **ORM**-инструмент (object-relational mapping - отображение данных на реальные объекты), который:

- Позволяет разработчикам .NET работать с базой данных с помощью объектов .NET.
- Устраняет потребность в большей части кода доступа к данным, который обычно необходимо писать.

Введение в EntityFramework Core

EF Core поддерживает работы с разными базами данных. Для этого используются соответствующие поставщики баз данных.

Подробнее о поставщиках БД – см. https://docs.microsoft.com/en-us/ef/core/providers/?tabs=dotnet-core-cli

Введение в EntityFramework Core

В EF Core доступ к данным осуществляется с помощью модели. Модель состоит из классов **сущностей** и объекта **контекста**, который предоставляет взаимодействие с базой данных.

Объект контекста позволяет запрашивать и сохранять данные.

Введение в EntityFramework Core

Базовые классы описаны в библиотеке ***Microsoft.EntityFrameworkCore***

Вспомогательные классы (например, для создания миграций) описаны в библиотеке ***Microsoft.EntityFrameworkCore.Tools***

Введение в EntityFramework Core

Поставщики данных для различных СУБД описаны в соответствующих библиотеках, например:

Microsoft.EntityFrameworkCore.SqlServer

Microsoft.EntityFrameworkCore.Sqlite

Pomelo.EntityFrameworkCore.MySql

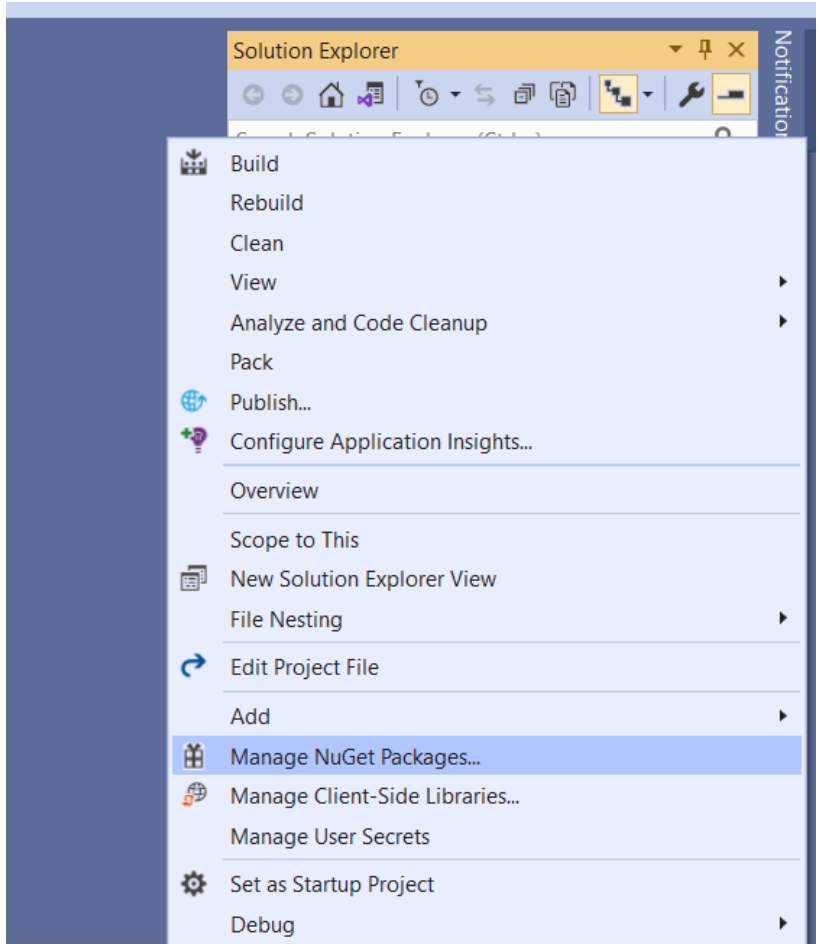
Npgsql.EntityFrameworkCore.PostgreSQL

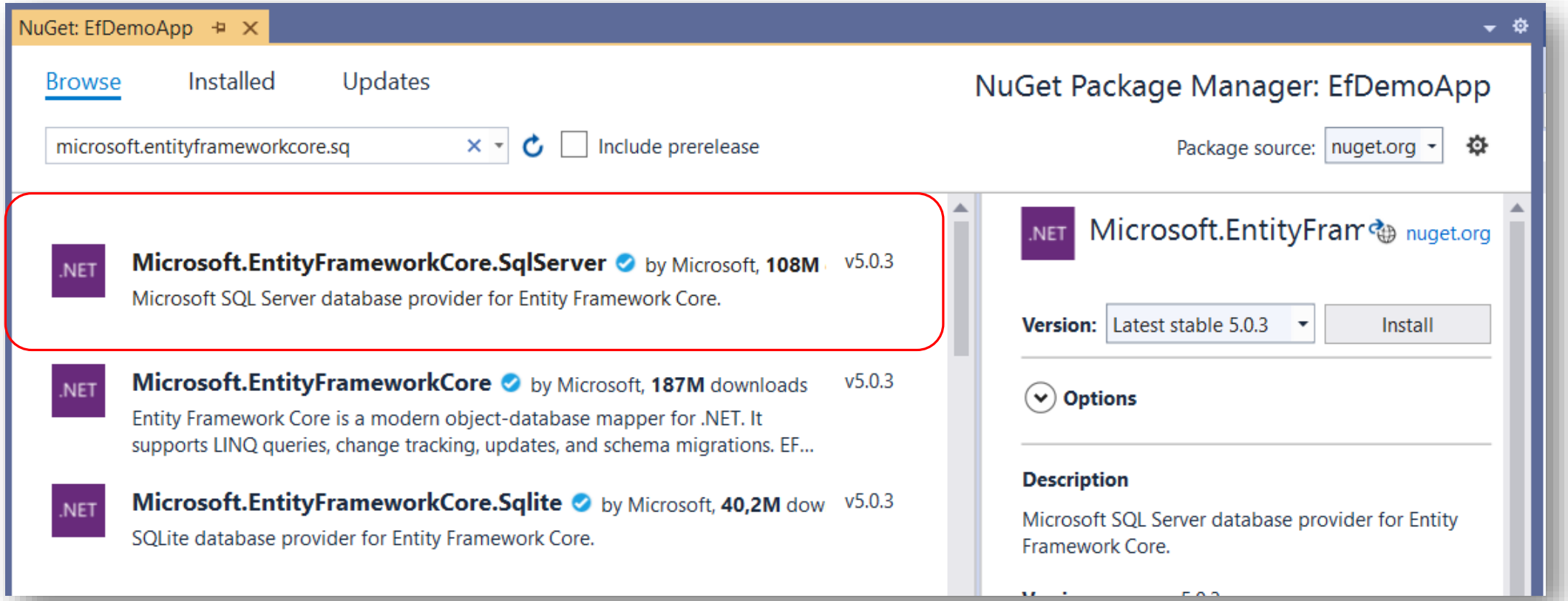
Введение в EntityFramework Core

EF поддерживает следующие подходы к разработке моделей:

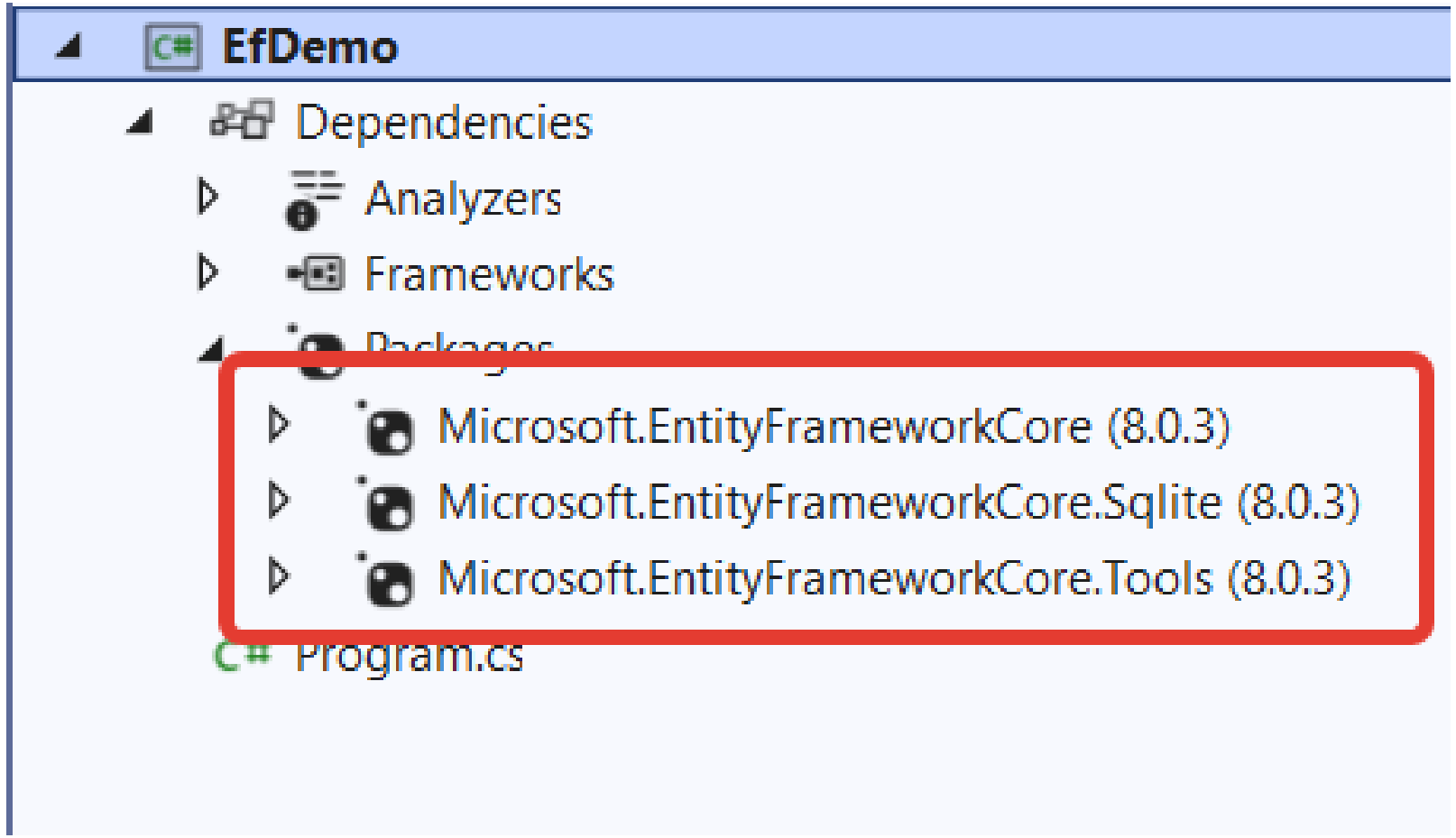
- Создание модели из существующей базы данных.
- Описание модели. Затем, используя EF Migrations создается база данных из модели. Миграции позволяют развивать базу данных по мере изменения модели.

Введение в EntityFramework Core




NuGet: EfDemoApp
Browse
Installed
Updates
NuGet Package Manager: EfDemoApp
microsoft.entityframeworkcore.sq
Include prerelease
Package source: nuget.org
.NET
Microsoft.EntityFrameworkCore.SqlServer by Microsoft, 108M
v5.0.3
Microsoft SQL Server database provider for Entity Framework Core.
.NET
Microsoft.EntityFrameworkCore by Microsoft, 187M downloads
v5.0.3
Entity Framework Core is a modern object-database mapper for .NET. It supports LINQ queries, change tracking, updates, and schema migrations. EF...
.NET
Microsoft.EntityFrameworkCore.Sqlite by Microsoft, 40,2M dow
v5.0.3
SQLite database provider for Entity Framework Core.
.NET
Microsoft.EntityFram
nuget.org
Version:
Latest stable 5.0.3
Install
Options
Description
Microsoft SQL Server database provider for Entity Framework Core.

Необходимые библиотеки



Описание сущностей

ВВЕДЕНИЕ В ENTITYFRAMEWORK CORE

Описание сущностей

```
public class Person
    {
        public int PersonId { get; set; }
        public string Name { get; set; }
      // навигационные свойства
        public int DepartmentId { get; set; }
        public Department Department { get; set; }
    }

    public class Department
    {
        public int Id { get; set; }
        public string Name { get; set; }
      // навигационные свойства
        public List<Person> Persons { get; set; }
    }
```

Описание контекста базы данных

ВВЕДЕНИЕ В ENTITYFRAMEWORK CORE

Контекст базы данных

Экземпляр **DbContext** предоставляет сеанс с базой данных и может использоваться для запроса и сохранения экземпляров сущностей.

DbContext - это комбинация шаблонов **Unit Of Work** и **Repository**.

Контекст базы данных

DbContext отслеживает изменения объектов при:

- получении объектов из запроса
- добавлении/изменении объекта

Привызове метода **SaveChanges** или **SaveChangesAsync** EF Core обнаруживает внесенные изменения и записывает их в базу данных.

Контекст базы данных

```
public class ApplicationDbContext : DbContext
{
    public DbSet<Person> Persons { get; set; }
    public DbSet<Department> Departments { get; set; }
}
```

Контекст базы данных

DbSet/DbSet<TEntity>: представляет набор объектов, которые отображаются на таблицы базы данных

Контекст базы данных

DbSet/DbSet<TEntity>: представляет набор объектов, которые отображаются на таблицы базы данных

DbSet позволяет делать выборку данных спомощью запросов LINQ.

EF Core преобразует эти запросы в запросы к БД, например, в запросы SQL

Конфигурирование контекста БД

ВВЕДЕНИЕ В ENTITYFRAMEWORK CORE

Конфигурирование контекста БД (вариант1)

```
public class ApplicationDbContext : DbContext
    {
        private string connectionString;
        public ApplicationDbContext(string connStr)
        {
            connectionString = connStr;
        }
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(connectionString);
        }
        public DbSet<Person> Persons { get; set; }
        public DbSet<Department> Departments { get; set; }
    }
```

Конфигурирование контекста БД (Вариант 2)

```
public class ApplicationDbContext : DbContext
{
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> opt)
                :base(opt)
          {

          }
        public DbSet<Person> Persons { get; set; }
        public DbSet<Department> Departments { get; set; }
}
```

Конфигурирование контекста БД (Вариант 2)

```
var connStr = @"Server=(localdb)\\mssqllocaldb; Database = EfDemo;"
                              + "Trusted_Connection = True; "
                              + "MultipleActiveResultSets = true";

var builder = new DbContextOptionsBuilder<ApplicationDbContext>();
var options = builder.UseSqlServer(connStr).Options;

var context = new ApplicationDbContext(options);
```

Ключевые поля

ВВЕДЕНИЕ В ENTITYFRAMEWORK CORE

Ключевые поля

По умолчанию свойство с именем Id или <имя типа> Id будет настроено как первичный ключ сущности.

```
public class Person
    {
        public int PersonId { get; set; }
        . . .
    }

public class Department
    {
        public int Id { get; set; }
        . . .
    }
```

Настройка ключевых полей в коде (аннотация данных)

```
using System.ComponentModel.DataAnnotations;
public class Person
    {
        [Key]
        public string PassNo { get; set; }
        . . .
    }
```

Настройка ключевых полей в коде (fluent API, в классе контекста БД)

```
protected override void OnModelCreating(ModelBuilder builder)
        {
            builder.Entity<Person>()
                .HasKey(p=>p.PassNo);
        }
```

Отношения между таблицами

ВВЕДЕНИЕ В ENTITYFRAMEWORK CORE

Отношения между таблицами (основные термины)

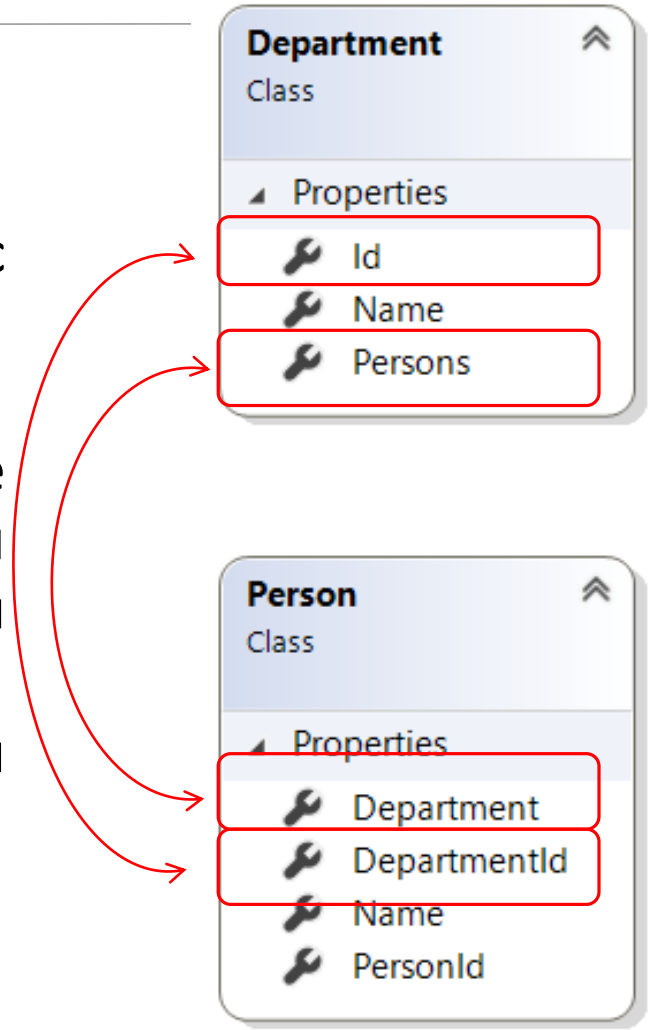
- **Dependent entity** (Зависимая сущность): это сущность, которая содержит свойства внешнего ключа. Иногда ее называют «child (дочерней)».
- **Principal entity** (Основная сущность): это сущность, которая содержит свойства первичного / альтернативного ключа. Иногда ее называют «parent (родительской)»
- **Primary Key** (Главный ключ): свойства, которые однозначно идентифицируют основную сущность.
- **Foreign Key** (Внешний ключ): свойства в зависимой сущности, которые используются для хранения значений основного ключа для связанной сущности
- **Navigation Property** (Свойство навигации): свойство, определенное для основной и / или зависимой сущности, которое ссылается на связанную сущность

Отношения между таблицами (основные термины)

- **Collection Navigation Property** (Свойство навигации коллекции): свойство навигации, которое содержит ссылки на множество связанных сущностей.
- **Reference navigation property** (Свойство навигации по ссылке): свойство навигации, которое содержит ссылку на один связанный объект.
- **Inverse navigation property** (Свойство обратной навигации): при обсуждении конкретного свойства навигации этот термин относится к свойству навигации на другом конце отношения.
- **Self-referencing relationship** (Отношение со ссылками на себя): отношение, в котором типы зависимой и основной сущностей совпадают.

Отношения между таблицами (один-ко-многим)

- **Department** – основная сущность (*Principal*)
- **Person** – зависимая сущность (*Dependent*)
- **Department.Id** – основной ключ (*Principal key*) – совпадает с главным ключом (*Primary key*)
- **Person.DepartmentId** – внешний ключ (*Foreign key*)
- **Person.Department** - Свойство навигации по ссылке (*Reference navigation property*), а также свойство обратной навигации (*Inverse navigation property*) для Department.Persons
- **Department.Persons** - Свойство навигации коллекции (*Collection Navigation Property*)



Отношения между таблицами (один-ко-многим)

Если EF Core обнаруживает навигационные свойства, и имена свойств соответствуют одному из правил:

<имя навигационного свойства>Id
<имя родительской сущности>Id

то связи и вторичные ключи в БД формируются автоматически

Настройка отношений вручную (Аннотация данных)

```
public class Person
    {
        . . .
        // навигационные свойства
        public int DepartmentId { get; set; }
        public Department Department { get; set; }
    }

    public class Department
    {
        public int Id { get; set; }
        public string Name { get; set; }
        // навигационные свойства
        [InverseProperty("Department")]
        public List<Person> Persons { get; set; }
    }
```

Настройка отношений вручную (fluent API)

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            modelBuilder.Entity<Person>()
                .HasOne(p => p.Department)
                .WithMany(d => d.Persons);
        }
```

Подключение контекста БД в ASP.Net Core

ВВЕДЕНИЕ В ENTITYFRAMEWORK CORE

Подключение контекста БД в ASP.Net Core

файл appsettings.json:

```
 "ConnectionStrings": {
     "Default":
"Server=(localdb)\\mssqllocaldb;   Database   =
EfDemo;     Trusted_Connection     =     True;
MultipleActiveResultSets = true"
   },
```

Подключение контекста БД в ASP.Net Core

```
services.AddDbContext<ApplicationDbContext>(opt => {
    opt.UseSqlServer(Configuration
                    .GetConnectionString("Default"));
            });
```

Подключение контекста БД в консольном приложении

```
var connString = "Data Source = Test.db";
var builder = Host.CreateApplicationBuilder();

builder.Services.AddDbContext<AppDbContext>(opt =>
     opt.UseSqlite(connString));

var host = builder.Build();

using var scope = host.Services.CreateScope();
using var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();

db.Database.EnsureDeleted();
await DbServices.Populate(db);

await host.RunAsync();
```

Инициализация данных

```
public static async Task Populate(AppDbContext appDbContext)
    {
        var result = appDbContext.Database.EnsureCreated();

       if(!appDbContext.Departments.Any())
        {
            var depts = appDbContext.Departments;
            for(int i = 1; i <= 3; i++)
            {
                await depts.AddAsync(new Department
                {
                     Name=$"Department {i}"
                });
            }
            await appDbContext.SaveChangesAsync();
        }
        if(!appDbContext.Persons.Any())
        {
            var persons = appDbContext.Persons;

            foreach(var dept in appDbContext.Departments)
            {
                for (int i = 1; i <= Random.Shared.Next(5, 10); i++)
                {
                    await persons.AddAsync(new Person
                    {
                        Name = $"Person {dept.Id}-{i}",
                        DepartmentId = dept.Id
                    });
                }
            }
            await appDbContext.SaveChangesAsync();
        }
    }
```

Печать данных

```
public static void ListAllData(AppDbContext db)
    {
        foreach(var dept in db.Departments)
        {
            Console.WriteLine(dept.Name);
            foreach(var person in db.Persons.Where(p=>p.DepartmentId.Equals(dept.Id)))
                Console.WriteLine($"\t{person.Name}");
        }
    }
```

Migrations

ВВЕДЕНИЕ В ENTITYFRAMEWORK CORE

Migrations

Миграции в EF Core предоставляет способ постепенного обновления схемы базы данных, чтобы поддерживать ее синхронизацию с моделью данных приложения, сохраняя при этом существующие данные в базе данных.

Migrations (добавление миграции)

Package Manager Console

Package source: All | Default project: EfDemoApp

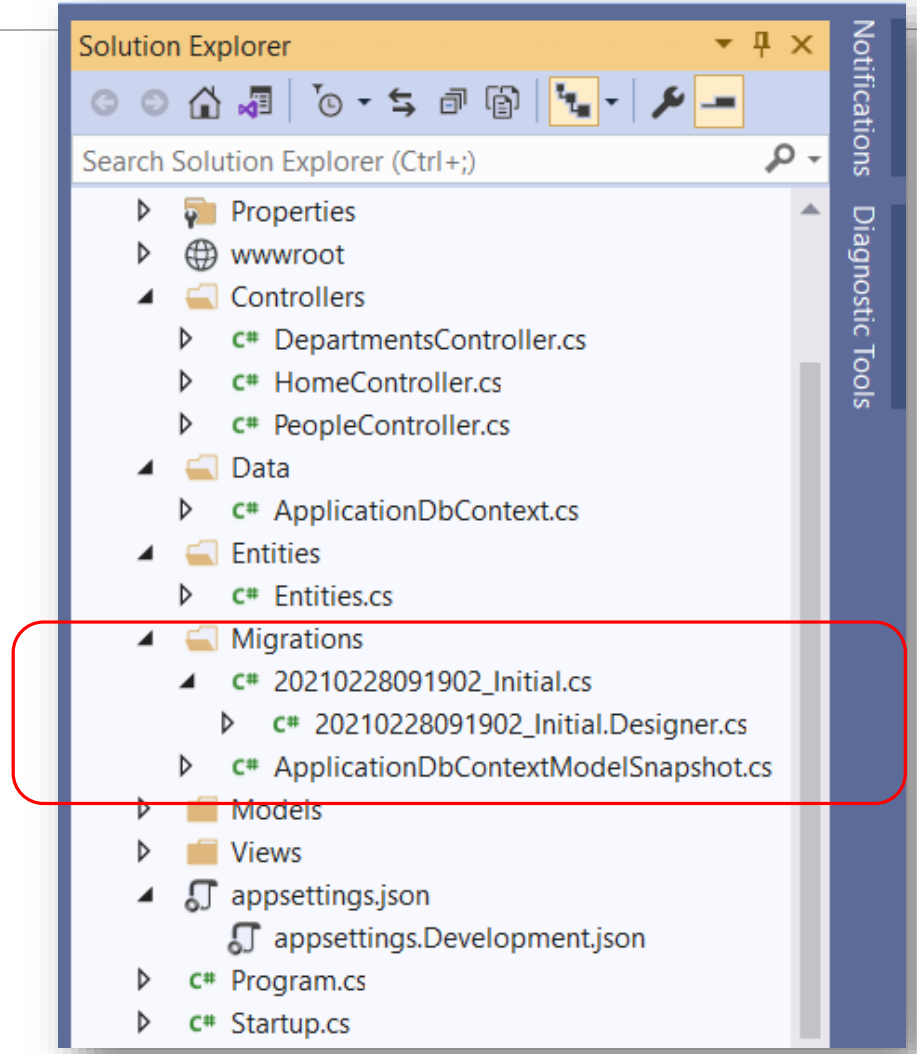
```
Type 'get-help NuGet' to see all available NuGet commands.

PM> add-migration Initial
```

186 %

Package Manager Console | Error List | Output

Migrations (добавление миграции)



Применение миграции

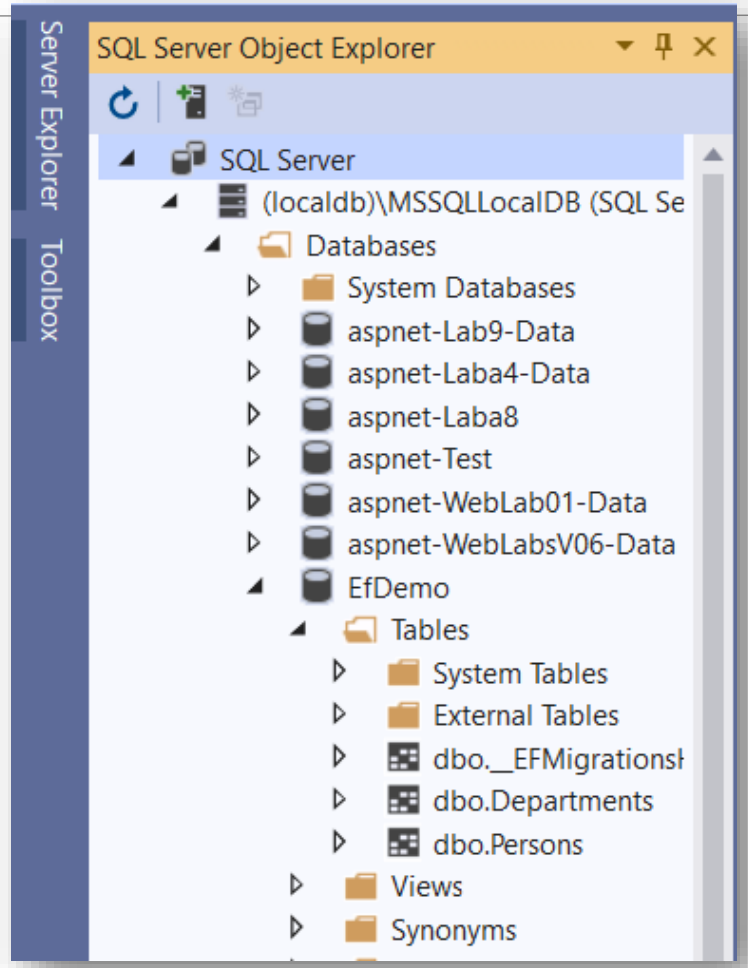
Package Manager Console

Package source: All Default project: EfDemoApp

```
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
```

186 %

Package Manager Console | Error List | Output


Server Explorer
Toolbox
SQL Server Object Explorer
SQL Server
(localdb)\MSSQLLocalDB (SQL Se
Databases
System Databases
aspnet-Lab9-Data
aspnet-Laba4-Data
aspnet-Laba8
aspnet-Test
aspnet-WebLab01-Data
aspnet-WebLabsV06-Data
EfDemo
Tables
System Tables
External Tables
dbo.__EFMigrationsH
dbo.Departments
dbo.Persons
Views
Synonyms

Добавление новой миграции

```
public class Person
    {
        public string PersonId { get; set; }
        public string Name { get; set; }
        public string Position { get; set; }
        // навигационные свойства
        public int DepartmentId { get; set; }
        public  Department  Department  {  get;
set; }
    }
```

Добавление новой миграции

Package Manager Console

Package source: All | Default project: EfDemoApp

```
Build started...
Build succeeded.
Done.
PM> add-migration position_added
```

186 %

Package Manager Console | Error List | Output

Package Manager Console

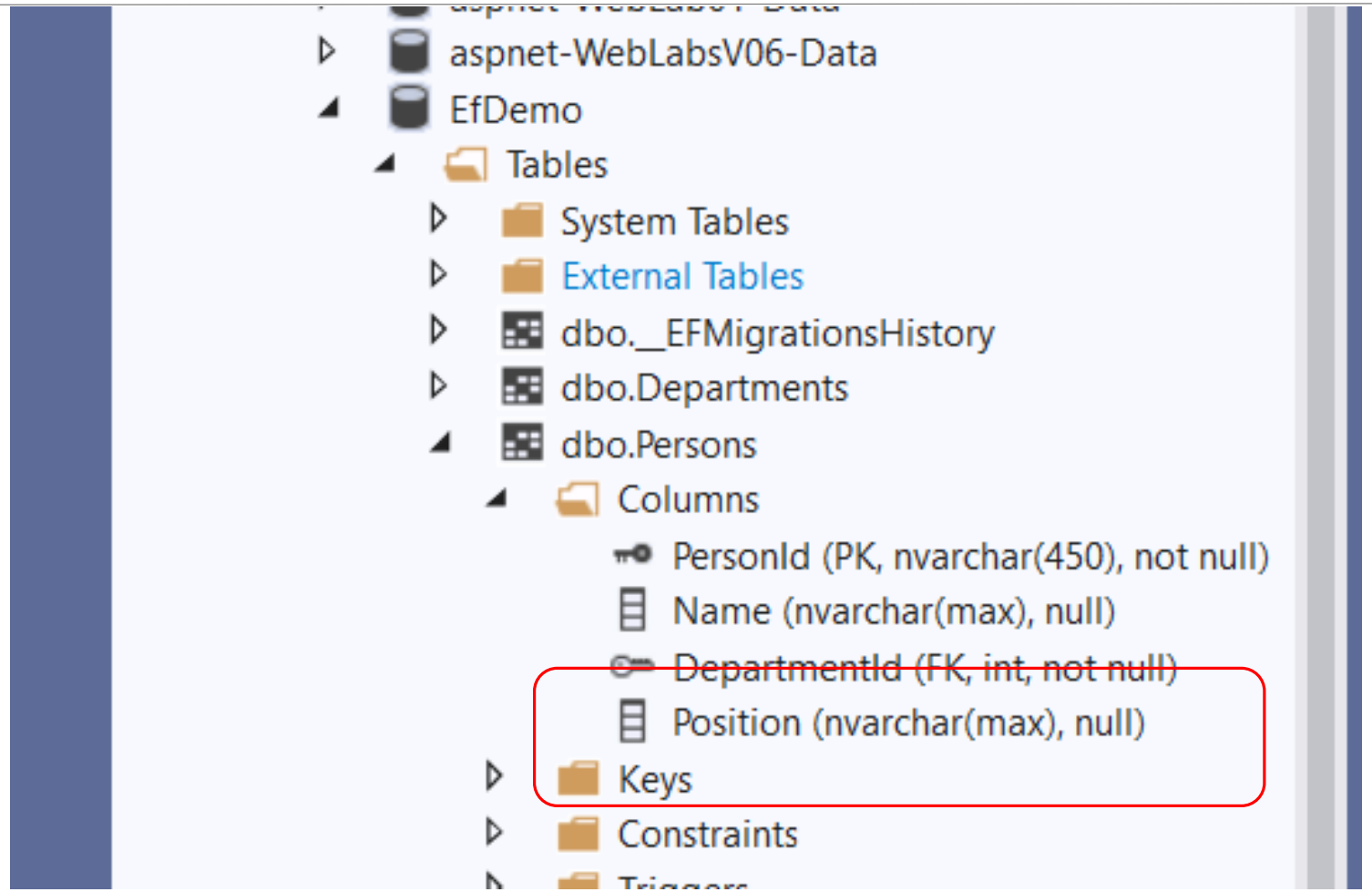
Package source: All | Default project: EfDemoAp

```
PM> update-database
```

186 %

Package Manager Console | Error List | Output

Добавление новой миграции



Запросы к контексту БД

ВВЕДЕНИЕ В ENTITYFRAMEWORK CORE

Entity Framework Core использует встроенный в язык запросов (LINQ) для запроса данных из базы данных. LINQ позволяет использовать C # (или любой другой язык .NET) для написания строго типизированных запросов.

EF Core передает запрос LINQ поставщику базы данных. Поставщики баз данных, в свою очередь, переводят его на язык запросов, специфичный для базы данных (например, SQL для реляционной базы данных).

Получение всех данных

```
var context = new ApplicationDbContext(options);
var persons = context.Persons.ToList();
```

Фильтрация данных

```
var persons = context.Persons
                .Where(p => p.Name == "Bob");
```

Поиск данных

```
var person = context.Persons
                .SingleOrDefault(p => p.Name == "Bob");
```

поиск по ключу

```
var person = context.Persons.Find(2);
```

Получение связанных данных

```
var persons = context.Persons
                .Include(p => p.Department);
```

Работа с контекстом

```
 var host = builder.Build();

using var scope = host.Services.CreateScope();
 using var db = scope.ServiceProvider.GetRequiredService<AppDbContext>()
     db.Database.EnsureDeleted();
     await DbServices.Populate(db);

    await db.Persons.AddAsync(new Person { Name = "Added1", DepartmentId = 2 });
     await db.Persons.AddAsync(new Person { Name = "Added2", DepartmentId = 2 });
     await db.Persons.AddAsync(new Person { Name = "Added3", DepartmentId = 2 });
     var person = await db.Persons.FindAsync(1);
     db.Persons.Remove(person);

 var person2 = new Person { PersonId = 2, Name = "ModifiedName", DepartmentId = 2 };
 db.Entry(person2).State = EntityState.Modified;
 Console.WriteLine("Data List");
await db.SaveChangesAsync();
 DbServices.ListAllData(db);
await host.RunAsync();
```

Ошибка

Работа с контекстом

```
var person2 = await db.Persons.FindAsync(2);
person2.Name = "ModifiedName";
person2.DepartmentId = 2;
```

Или

```
var person2 = await db.Persons
                .Where(p=>p.PersonId==2)
                .ExecuteUpdateAsync(setter=>setter
                    .SetProperty(p=>p.Name, "ModifiedName")
                    .SetProperty(p=>p.DepartmentId, 2)
                );
```

Сласс DemoActions

```
public static async Task UpdateDb(DbContextOptions<AppDbContext> options)
    {
        using var db = new AppDbContext(options);

       db.Database.EnsureDeleted();
       await DbServices.Populate(db);

       await db.Persons.AddAsync(new Person { Name = "Added1", DepartmentId = 2 });
        await db.Persons.AddAsync(new Person { Name = "Added2", DepartmentId = 2 });
        await db.Persons.AddAsync(new Person { Name = "Added3", DepartmentId = 2 });
        var person = await db.Persons.FindAsync(1);
        db.Persons.Remove(person);

       await db.SaveChangesAsync();
   }
```

Сласс DemoActions

```
public static async Task StateDemo(DbContextOptions<AppDbContext> options)
    {
        using var db = new AppDbContext(options);

       var person = new Person { PersonId = 2, Name = "ModifiedName", DepartmentId = 3 };

        db.Entry(person).State = EntityState.Modified;
        await db.SaveChangesAsync();

    }
```

Сласс DemoActions

```
public static async Task ExecuteUpdateDemo(DbContextOptions<AppDbContext> options)
    {
        using var db = new AppDbContext(options);
        await db.Persons
                .Where(p=>p.PersonId>2&&p.DepartmentId==1)
                .ExecuteUpdateAsync(setter=>setter
                    .SetProperty(p=>p.Name, p=> "ModifiedName-"+p.PersonId)
                    .SetProperty(p=>p.DepartmentId, 2)
                );

    }
```

```
 var connString = "Data Source = Test.db";

builder.Services.AddDbContext<AppDbContext>(opt =>
     opt.UseSqlite(connString));

var options = new DbContextOptionsBuilder<AppDbContext>()
     .UseSqlite(connString)
     .Options;

await DemoActions.UpdateDb(options);

//await DemoActions.StateDemo(options);
 await DemoActions.ExecuteUpdateDemo(options);

DbServices.ListAllData(new AppDbContext(options));
```